

#JONSECOPS

AWS CLOUD PROJECT

-CLOUD IDENTITIY COMPROMISE INVESTIGATION-

Table of Contents

EXECUTIVE SUMMARY: 3

PHASE 1: ATTACK SETUP 4

- CONFIGURING S3 BUCKET SETTINGS 4
- CREATING CLOUDTRAIL TRAIL 5
- CREATING COMPROMISED IAM USER 6

PHASE 2: SIMULATING THE ATTACK VIA CLOUDSHELL 8

- CONFIGURING THE ATTACKER PROFILE 8
- CONFIGURING AMAZON ATHENA 9

PHASE 3: FORENSIC ANALYSIS 10

- CREATING EXTERNAL TABLE 10
- FORENSIC QUERY 11
- FORENSIC TIMELINE 12

PHASE 4: RECOMMENDATIONS 13

- Appendix: PICERL Incident Response Lifecycle Matrix 14

7339 86D1 JSO JonSecOps

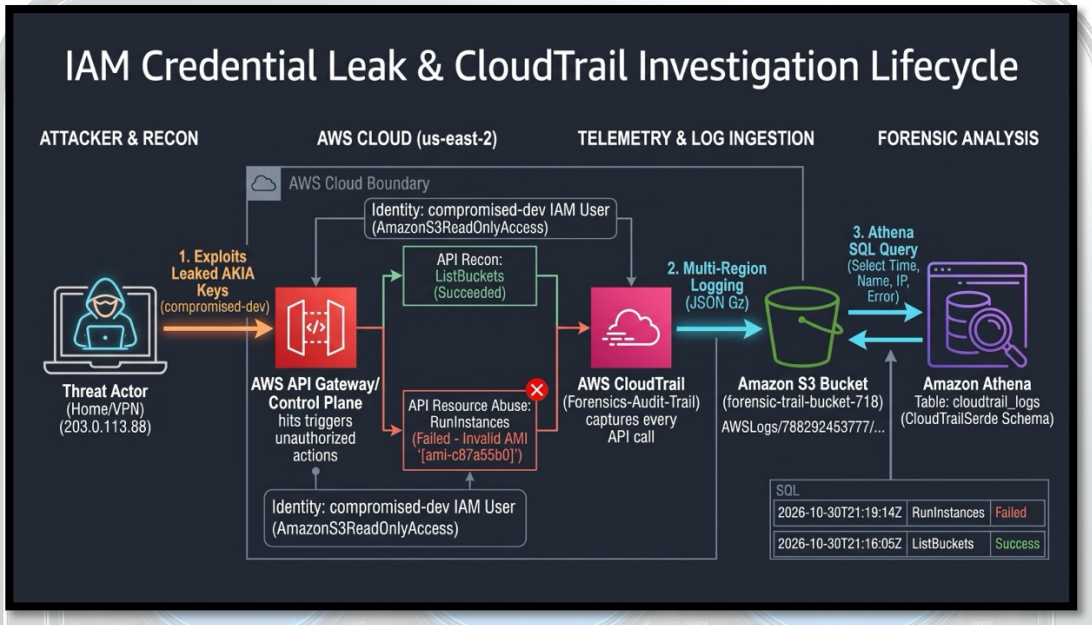
EXECUTIVE SUMMARY:

This report details a cloud forensic investigation concerning the simulated exposure of long-term AWS programmatic access keys. A threat actor leveraged the compromised IAM identity (**compromised-dev**) to perform early reconnaissance against the environment's storage architecture (**ListBuckets**) and attempted unauthorized resource abuse via **EC2** compute provisioning (**RunInstances**).

By configuring AWS CloudTrail telemetry and deploying Amazon Athena with a native CloudTrailSerde schema, the investigation successfully parsed gzipped, nested JSON logs to isolate the threat actor's identity, source IP, and chronological API timeline. The failed compute provisioning attempt (rejected due to an invalid AMI ID) was successfully documented, demonstrating robust cloud native auditing capabilities.



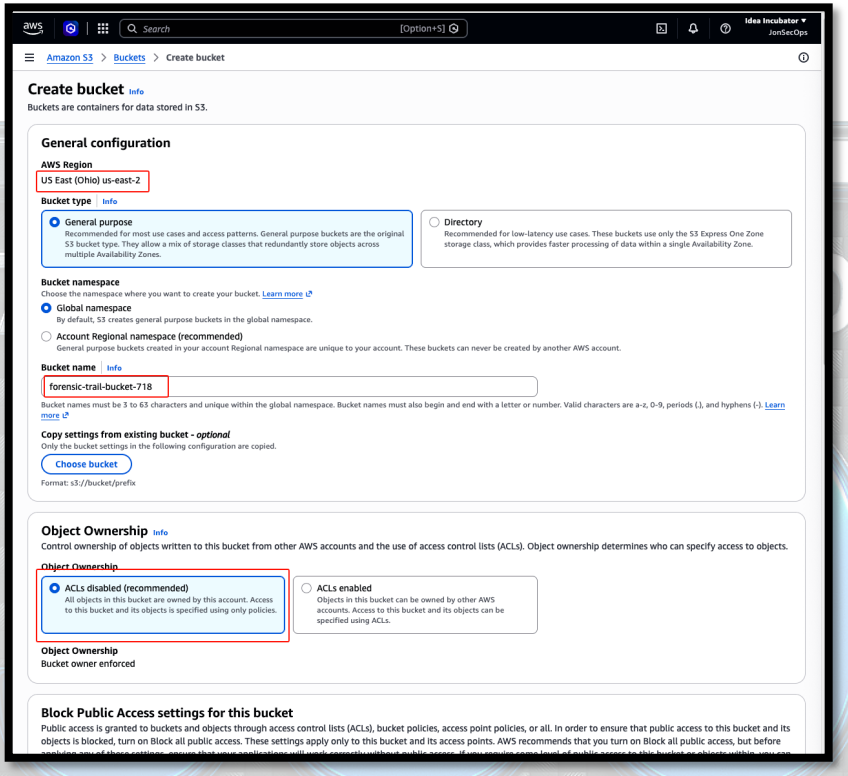
PHASE 1: ATTACK SETUP



CONFIGURING S3 BUCKET SETTINGS

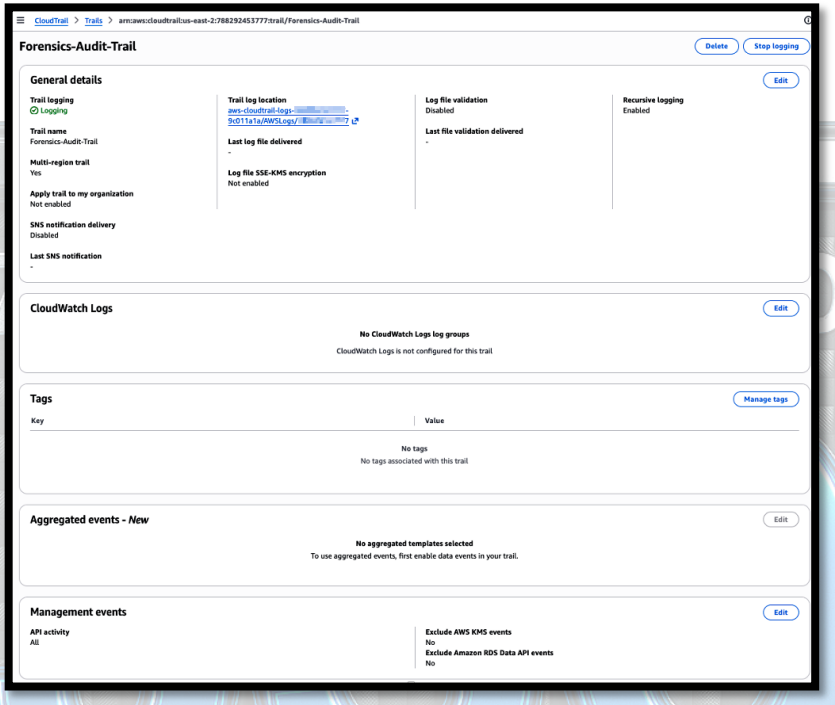
To properly collect and secure the control plane audit logs before the attack simulation, an Amazon **S3** evidence bucket named **forensic-trail-bucket-718** was created. The bucket was configured with strict object ownership rules and "Block all public access" enabled to ensure the integrity and confidentiality of the forensic artifacts.

JonSecOps



CREATING CLOUDTRAIL TRAIL

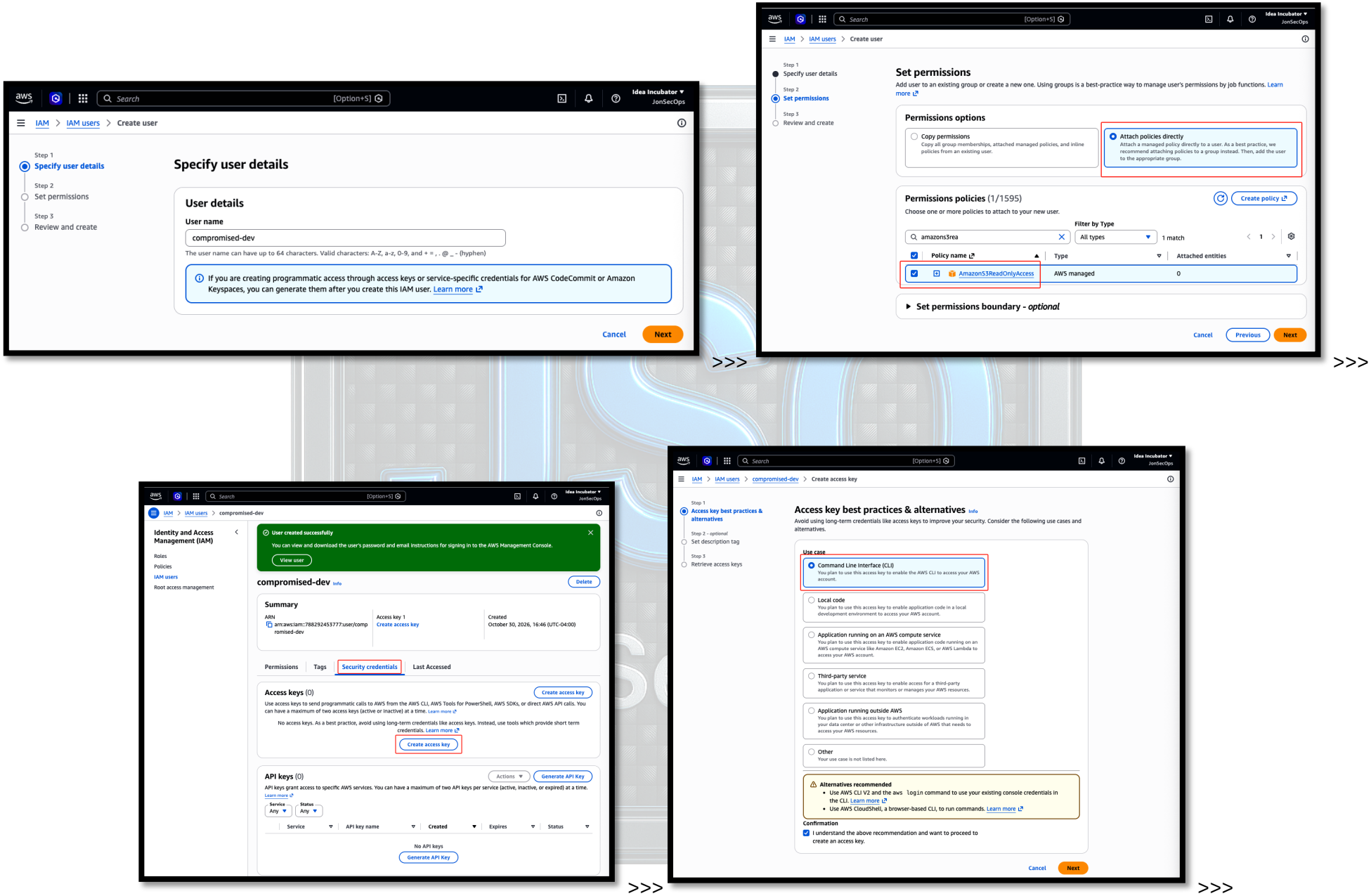
To monitor management-plane activity, AWS **CloudTrail** named **Forensics-Audit-Trail** was configured as an immutable audit log, actively capturing all multi-region read/write API events and seamlessly delivering the gzipped JSON telemetry to the designated S3 evidence bucket.

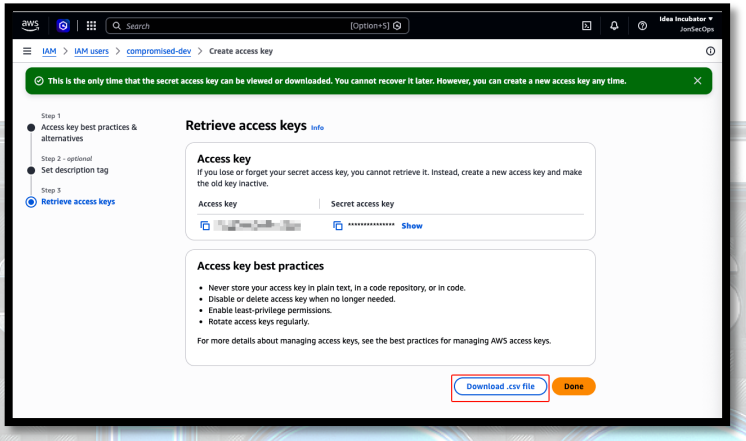


CREATING COMPROMISED IAM USER

To simulate the threat actor's entry point, a dedicated **IAM** user named **compromised-dev** was provisioned with programmatic access keys for Command Line Interface (CLI) execution. The identity was specifically assigned the **AmazonS3ReadOnlyAccess** managed policy to safely demonstrate reconnaissance capabilities while preventing uncontrolled modification of the AWS environment.

JonSecOps

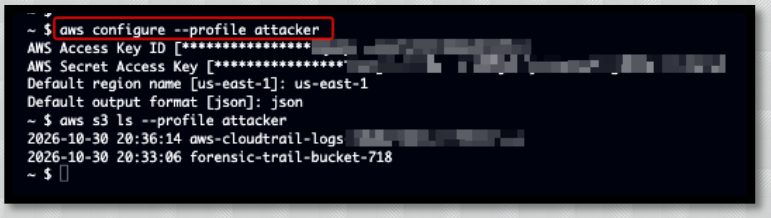

Search
[Option+S]
Idea Incubator
JonSecOps
IAM > IAM users > Create user
Step 1
Specify user details
Step 2
Set permissions
Step 3
Review and create
Specify user details
User details
User name
compromised-dev
The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)
If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. Learn more
Cancel
Next
>>>
Set permissions
Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. Learn more
Permissions options
Copy permissions
Copy all group memberships, attached managed policies, and inline policies from an existing user.
Attach policies directly
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.
Permissions policies (1/1595)
Choose one or more policies to attach to your new user.
Create policy
Filter by Type
amazons3rea
All types
1 match
Policy name
Type
Attached entities
AmazonS3ReadOnlyAccess
AWS managed
0
Set permissions boundary - optional
Cancel
Previous
Next
>>>
IAM > IAM users > compromised-dev
Identity and Access Management (IAM)
Roles
Policies
IAM users
Root access management
User created successfully
You can view and download the user's password and email instructions for signing in to the AWS Management Console.
View user
compromised-dev Info
Delete
Summary
ARN
arn:aws:iam::788292453777:user/compromised-dev
Access key 1
Create access key
Created
October 30, 2026, 16:46 (UTC-04:00)
Permissions
Tags
Security credentials
Last Accessed
Access keys (0)
Create access key
Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. Learn more
No access keys. As a best practice, avoid using long-term credentials like access keys. Instead, use tools which provide short term credentials. Learn more
Create access key
API keys (0)
Actions
Generate API Key
API keys grant access to specific AWS services. You can have a maximum of two API keys per service (active, inactive, or expired) at a time. Learn more
Service
Any
Status
Any
Service
API key name
Created
Expires
Status
No API keys
Generate API Key
>>>
IAM > IAM users > compromised-dev > Create access key
Step 1
Access key best practices & alternatives
Step 2 - optional
Set description tag
Step 3
Retrieve access keys
Access key best practices & alternatives Info
Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.
Use case
Command Line Interface (CLI)
You plan to use this access key to enable the AWS CLI to access your AWS account.
Local code
You plan to use this access key to enable application code in a local development environment to access your AWS account.
Application running on an AWS compute service
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.
Third-party service
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.
Application running outside AWS
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.
Other
Your use case is not listed here.
Alternatives recommended
Use AWS CLI V2 and the aws login command to use your existing console credentials in the CLI. Learn more
Use AWS CloudShell, a browser-based CLI, to run commands. Learn more
Confirmation
I understand the above recommendation and want to proceed to create an access key.
Cancel
Next
>>>



PHASE 2: SIMULATING THE ATTACK VIA CLOUDSHELL

CONFIGURING THE ATTACKER PROFILE

To execute the attack simulation, AWS **CloudShell** was utilized as a secure, browser-based terminal. An attacker profile was configured using the leaked programmatic access keys (AKIA...), allowing the threat actor to authenticate to the management plane from an external context and systematically execute reconnaissance and resource abuse API calls.

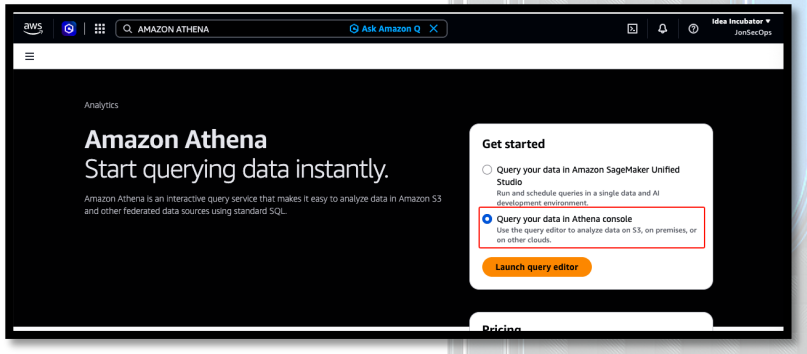


```
~ $ aws configure --profile attacker
AWS Access Key ID [****************
AWS Secret Access Key [****************
Default region name [us-east-1]: us-east-1
Default output format [json]: json
~ $ aws s3 ls --profile attacker
2026-10-30 20:36:14 aws-cloudtrail-logs-788292453777-9c011a1a
2026-10-30 20:33:06 forensic-trail-bucket-718
~ $ aws ec2 run-instances --image-id ami-c87a55b0 --instance-type p2.xlarge --profile attacker

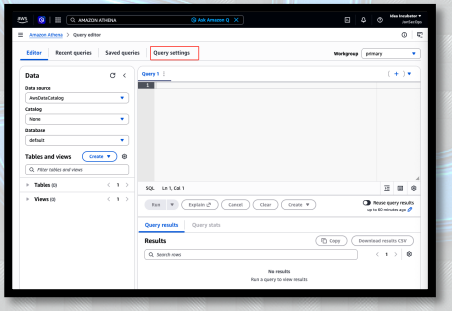
aws: [ERROR]: An error occurred (InvalidAMIID.NotFound) when calling the RunInstances operation: The image id '[ami-c87a55b0]' does not exist
~ $
```

CONFIGURING AMAZON ATHENA

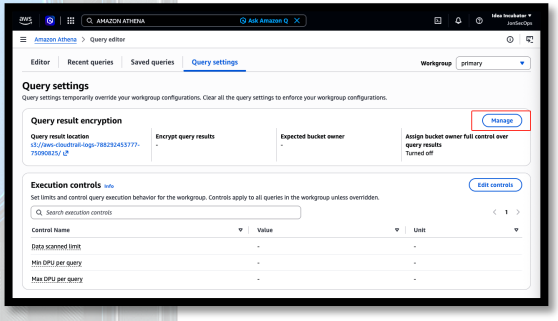
Because raw CloudTrail telemetry is delivered to S3 as heavily nested, gzipped JSON arrays, Amazon Athena was deployed to provide a robust, serverless SQL query engine. An external table schema named **cloudtrail_logs** was constructed utilizing AWS's native **CloudTrailSerde** parser, allowing investigators to accurately flatten and query the massive log partitions to reconstruct a chronological timeline of the threat actor's activity.



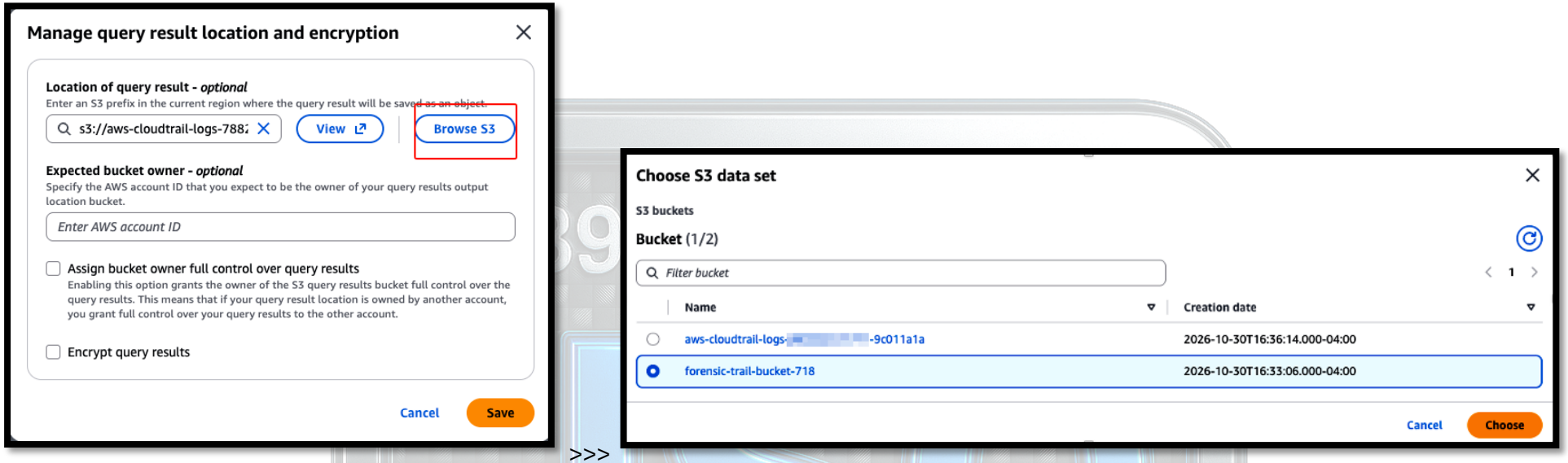
>>>



> >>



>>>



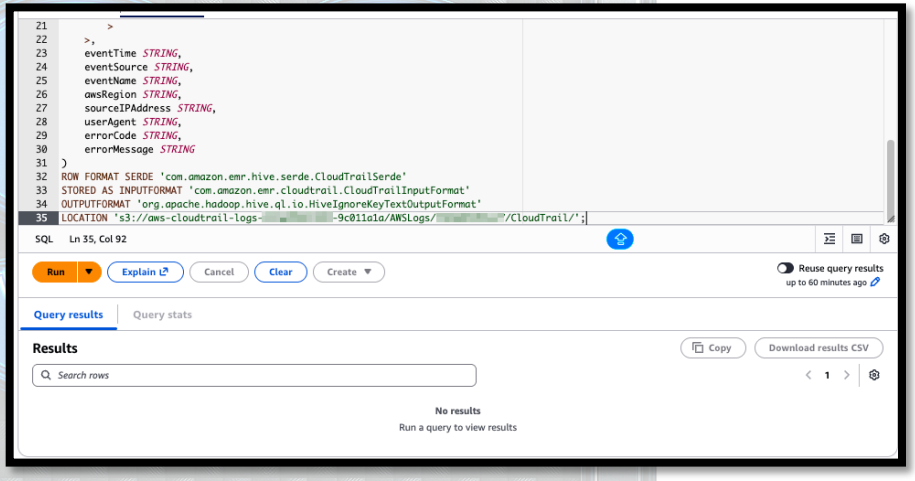
PHASE 3: FORENSIC ANALYSIS

CREATING EXTERNAL TABLE

To parse and analyze multi-region CloudTrail JSON logs stored within the Amazon S3 evidence bucket, an external table schema (cloudtrail_logs) was created in Amazon Athena. Utilizing AWS's native CloudTrailSerde parser and structured input formatting, the environment successfully mapped the gzipped, nested JSON log partitions to enable standard SQL-based forensic queries.

JonSecOps

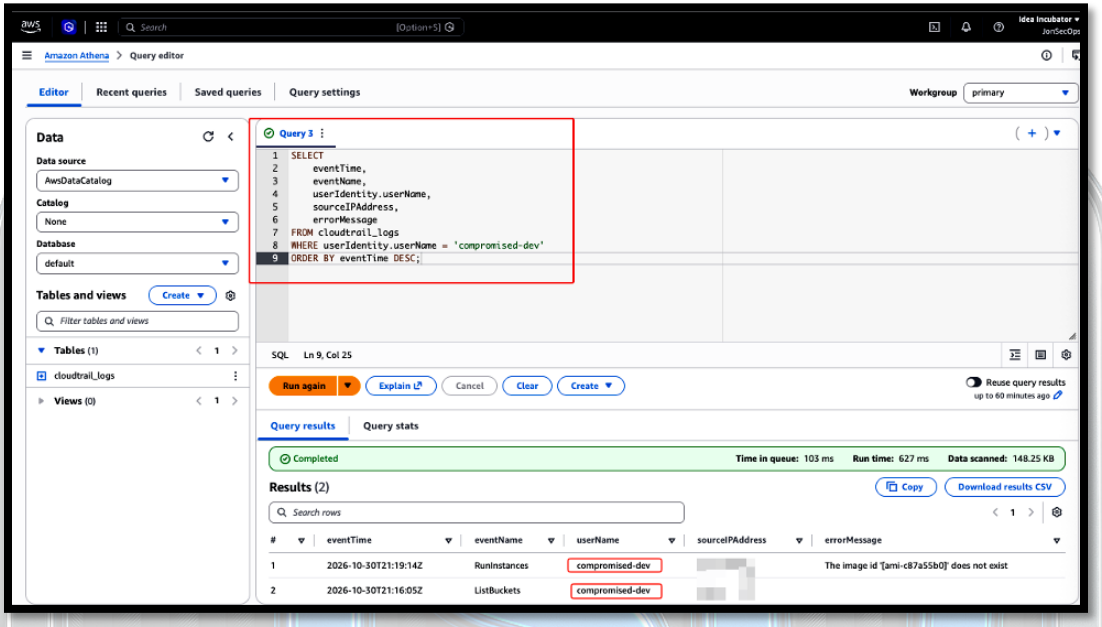
```sql
CREATE EXTERNAL TABLE cloudtrail_logs (
    eventVersion STRING,
    userIdentity STRUCT<
        type: STRING,
        principalId: STRING,
        arn: STRING,
        accountId: STRING,
        invokedBy: STRING,
        accessKeyId: STRING,
        userName: STRING,
        sessionContext: STRUCT<
            attributes: STRUCT<
                mfaAuthenticated: STRING,
                creationDate: STRING>,
            sessionIssuer: STRUCT<
                type: STRING,
                principalId: STRING,
                arn: STRING,
                accountId: STRING,
                userName: STRING>
        >
    >,
    eventTime STRING,
    eventSource STRING,
    eventName STRING,
    awsRegion STRING,
    sourceIPAddress STRING,
    userAgent STRING,
    errorCode STRING,
    errorMessage STRING
)
ROW FORMAT SERDE 'com.amazon.emr.hive.serde.CloudTrailSerde'
STORED AS INPUTFORMAT 'com.amazon.emr.cloudtrail.CloudTrailInputFormat'
OUTPUTFORMAT 'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION 's3://aws-cloudtrail-logs-[redacted]-9c011a1a/AWSLogs/[redacted]/C
```



FORENSIC QUERY

Following table creation and partition repair (MSCK REPAIR TABLE), targeted SQL queries were executed against the log repository. The investigation successfully isolated the threat actor's control-plane activity originating from the compromised IAM identity (compromised-dev), capturing unauthorized reconnaissance (**ListBuckets**) and failed resource abuse (**RunInstances**) within the chronological event timeline.

86D1
JonSecOps



FORENSIC TIMELINE

1. **ListBuckets** (at 2026-10-30T21:16:05Z):
   - This is the attacker performing early **reconnaissance** using the leaked access keys (**compromised-dev**). They are listing all S3 buckets in the AWS account to see what data or evidence is available to plunder.
2. **RunInstances** (at 2026-10-30T21:19:14Z):
   - This is the attacker attempting **resource abuse** (trying to spin up a GPU or high-powered EC2 instance, likely for cryptomining).
   - The **errorMessage** column clearly shows: *"The image id '[ami-c87a55b0]' does not exist"*. This is because the AMI ID used in the simulation script was invalid or didn't exist in that region, causing the attack to fail.

PHASE 4: RECOMMENDATIONS

- **Implement Network Flow Telemetry:** While CloudTrail successfully identified the management-plane API calls used for reconnaissance and compute provisioning, it does not record individual S3 file access or track byte-level data transfers. To accurately detect data exfiltration, VPC Flow Logs and S3 Object-Level Data Events must be enabled.

- **Establish Automated Containment Pipelines:** Manual SQL querying via Amazon Athena is highly effective for post-incident forensics but lacks real-time prevention. The environment should transition toward an event-driven architecture, utilizing Amazon GuardDuty and AWS EventBridge to automatically revoke compromised IAM roles upon detecting anomalous API behavior.

- **Enforce Short-Term Credentials:** The simulated compromise succeeded because long-term programmatic access keys (AKIA...) were utilized. Security policies should be updated to strictly enforce the use of short-term, dynamically generated credentials (via AWS STS) for all developer access and automated workloads.

JSO
JonSecOps

Appendix: PICERL Incident Response Lifecycle Matrix

IR Phase	Actions Executed	Key Findings & Outcomes
Preparation	Configured an S3 evidence bucket with strict access controls and deployed a multi-region AWS CloudTrail.	Ensured that immutable management-plane telemetry and API activity were securely captured prior to the incident.
Identification	Deployed Amazon Athena with a CloudTrailSerde schema to parse complex, gzipped JSON logs via standard SQL.	Successfully identified unauthorized reconnaissance (ListBuckets) and resource abuse attempts (RunInstances) by the compromised-dev identity.
Containment	Traced the chronological timeline to pinpoint the exact compromised programmatic access keys (AKIA...) and source IP address.	Scoped the active threat, providing the exact IAM identifiers required for immediate credential revocation and network blocking.
Eradication	Invalidated the exposed access keys and removed the vulnerable identity from the environment.	Eliminated the threat actor's management-plane foothold by deleting the compromised-dev IAM user.
Recovery	Analyzed the Athena errorMessage logs to verify the success or failure of the threat actor's actions.	Confirmed the compute provisioning failed gracefully due to an invalid AMI ID, verifying no rogue infrastructure required teardown.
Lessons Learned	Synthesized the management-plane findings to improve future defensive posture and automated response.	Identified the critical need to transition to short-term STS credentials and implement automated containment via EventBridge and GuardDuty.